

Software Engineering

Lecture 11

Software Engineering

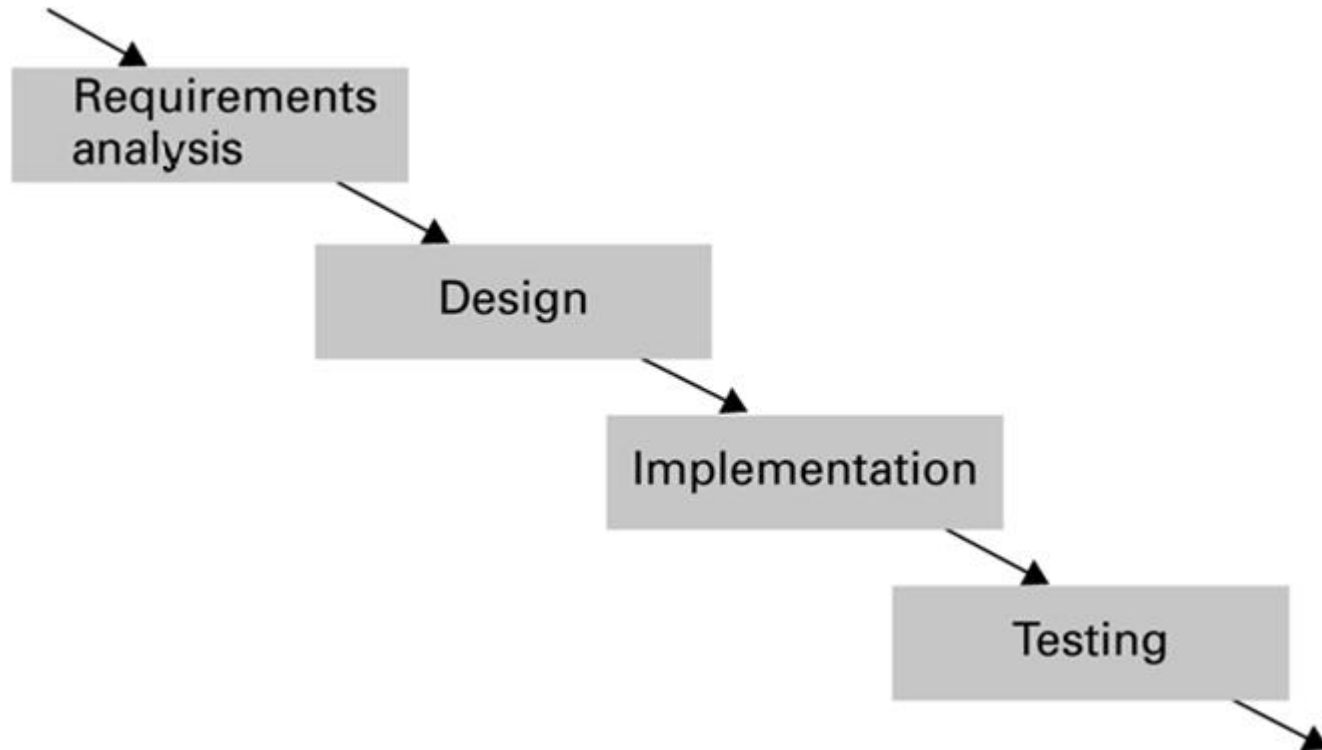
- The Software Engineering Discipline
- The Software Life Cycle
- Software Engineering Methodologies
- Modularity
- Tools of the Trade
- Testing
- Documentation
- Software Ownership and Liability

The Software Engineering Discipline

- Distinct from other engineering fields
 - Prefabricated components
 - Metrics
- Practitioners versus Theoreticians
- Professional Organizations: ACM, IEEE, etc.
 - Codes of professional ethics
 - Standards

	Traditional Engineering	Software Engineering
“Off-the-shelf” parts available	Often	Sometimes
Required Performance	Within tolerance	Perfect
Quality Metrics	Mean time between Failure (MTBF)	Unclear
Scientific Basis	Physics	Unclear

The development phase of the software life cycle



Requirements Analysis

- Based on application user requirements and some technical specifications
 - In **bespoke software development**, user is a concrete entity which is ordering the development of software, and the requirements must be collected from this entity directly.
 - In **commercial off-the-shelf software development**, user is less precisely defined and market must be studied by the developer himself.
- Performs a feasibility study
- The output of the analysis stage is the “**software requirements specification document**”

Design Stage

- The specification stage concentrates on **what** the system should do, while the design stage concentrates on **how** the system will accomplish those goals
- User interface design requires mixed knowledge of arts, psychology, ergonomics and programming.
- There are standardized notational systems and numerous modelling and diagramming methodologies for designing the software.

Implementation Stage

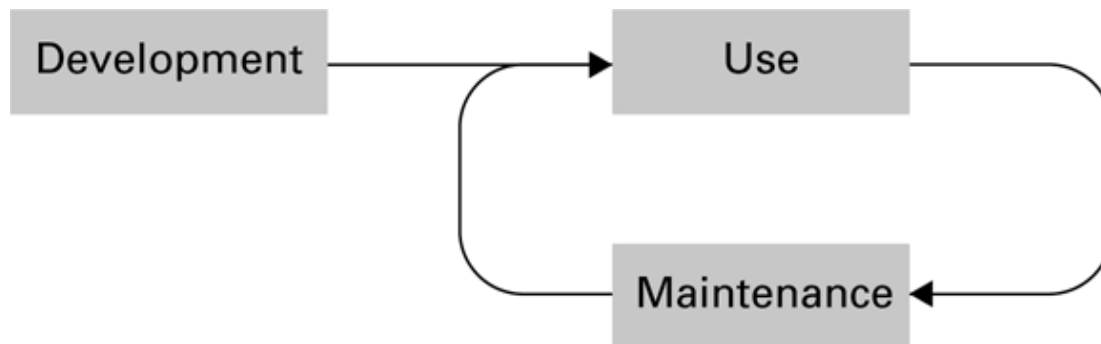
- Implementation Stage
 - Write programs
 - Create data files
 - Develop databases
- A software analyst (software architect) is involved in software development at the specification and design stage
- A programmer writes the program to implement given design.
- In practice, software architects are usually very experienced programmers.

Testing Stage

- Testing occurs in two forms:
 - Validation testing – checking to see if the system meets the original requirements and specifications
 - Defect testing – identifying and correcting errors (bugs) of the system
- Granularity of testing:
 - Unit testing – test a smallest software module:
 - For imperative programming, it is often functions
 - For OO, it is often class
 - Test codes are written to call the function with input at boundary and/or singular points
 - To test a module in a system, simplified versions of other modules (called **stubs or mock objects**) in the system are often used
 - Integration testing – test how different modules integrate together
 - System testing – test the entire system as a whole

The software life cycle

- After development process finished and the software has been deployed for use, continuous maintenance is required.
- Maintenance is about fixing bugs, updating the software with new features or sometime completely rewriting the software from scratch.



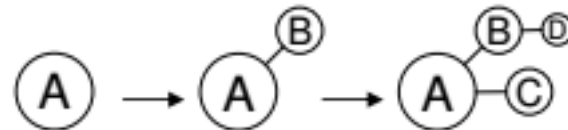
Software Development Methodologies

❑ Waterfall model

- Strictly following the orders of the four stages of software development to design a complete system from scratch
- Too slow to react to a dynamic environment

❑ Incremental model

- Start with a simplified system, and incrementally build more-and-more complete systems



❑ Iterative model

- Start with a full system with simplified modules, and incrementally build more-and-more complete modules



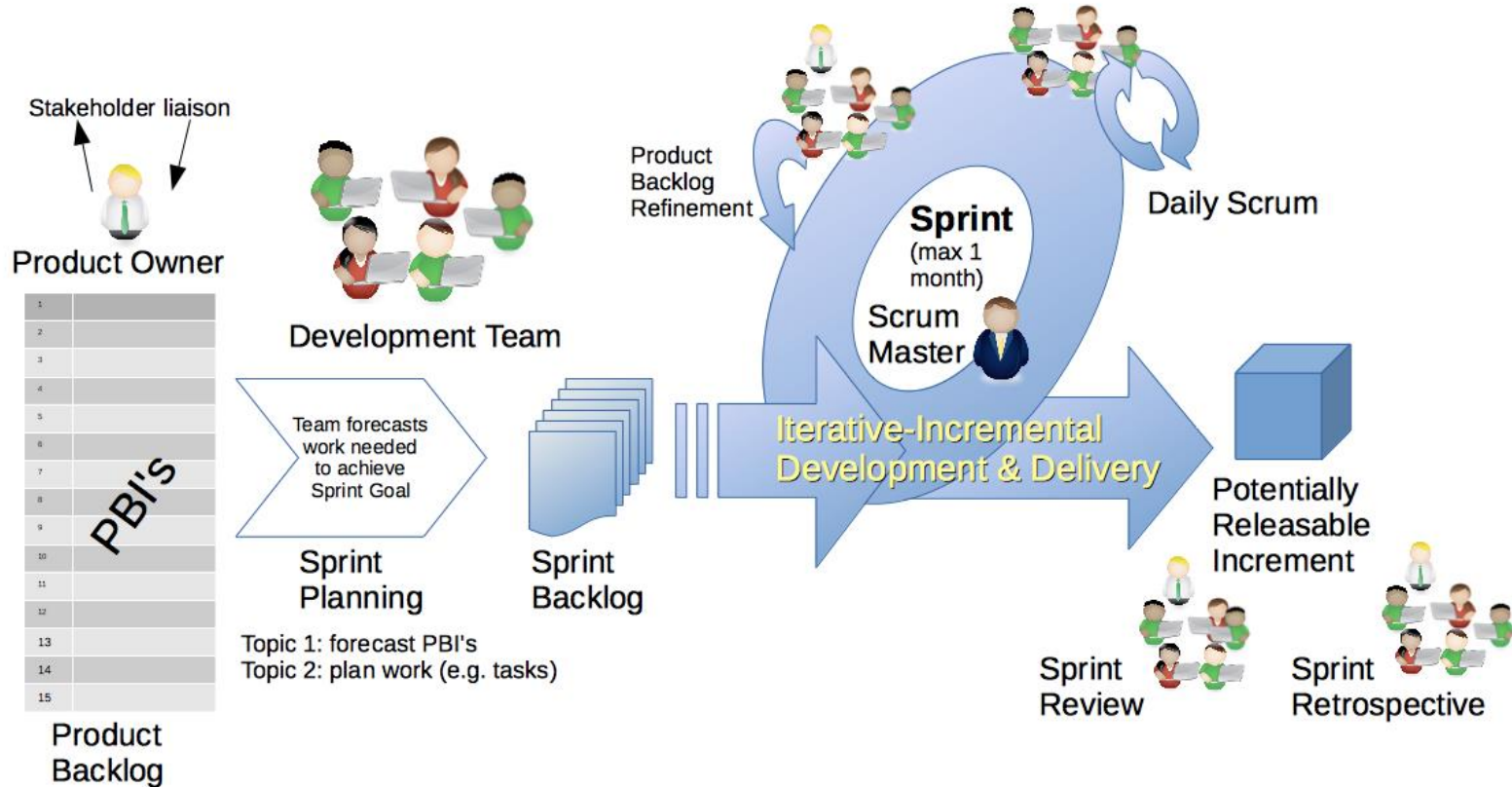
Rapid Applications Development

- For incremental and iterative development models, the early versions of the final system is often called **prototypes**
 - Throwaway prototyping – the prototypes are only used for communication and quick verification of the design; later, a fresh implementation will be used for the real product
 - Evolutionary prototyping – the prototypes will be refined incrementally and eventually become the final system
- Agile Methodologies:
 - Extreme Programming – less formal, less paperwork, more open and free set of principles and practices such as:
 - Pair programming
 - Planning game: exploration, commitment and steering phases
 - Test-driver development
 - Shared understanding: coding standards, collective ownership
 - Scrum – has its own jargon and workflow determining exactly how each step must be carried out. For ex, sprint backlog, burn-down charts, daily scrum etc.

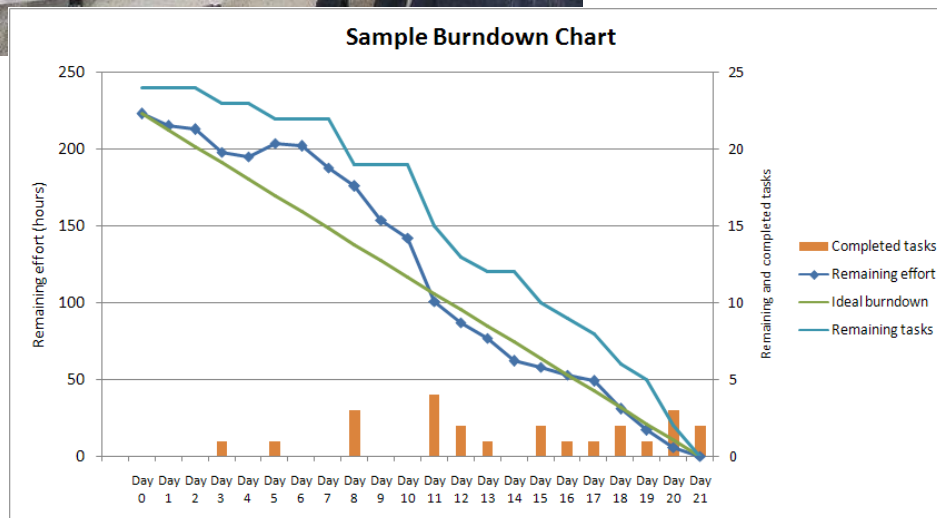
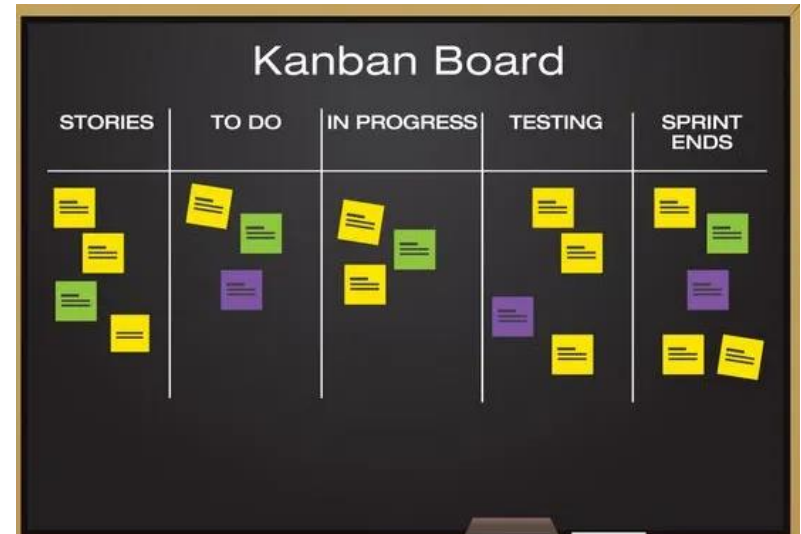
Scrum Roles

- **Product owner** - stakeholder, voice of customers
 - writes user stories
 - adds product backlogs and prioritized them
 - demonstrates solution to key stakeholders
 - announces releases
 - updates statuses on backlogs
- **Development Team** - with every *sprint* development team delivers new feature.
- **Scrum Master** - plays the role of a buffer between Product Owner and Development Team, and ensures that Scrum framework is followed.

Scrum Framework



Scrum Sprint Backlog and Burndown Chart



Open Source Software Development

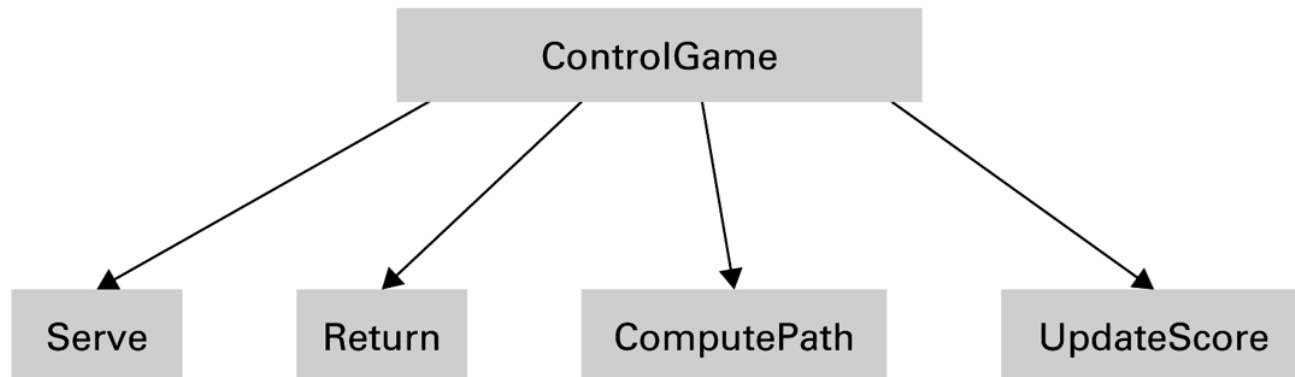
- Software developed by programming community for free
- **Format:**
 - single author writes an initial version of the software, usually to fulfill his/her own needs, and posts the source code with documentation on Internet
 - Others can download it and use it without any change
 - They can also further modify/enhance the software to fit their own needs, or fix the bugs they found.
 - Users then report about their changes to the original source code to the author, and author updates his posted source code.

Modularity

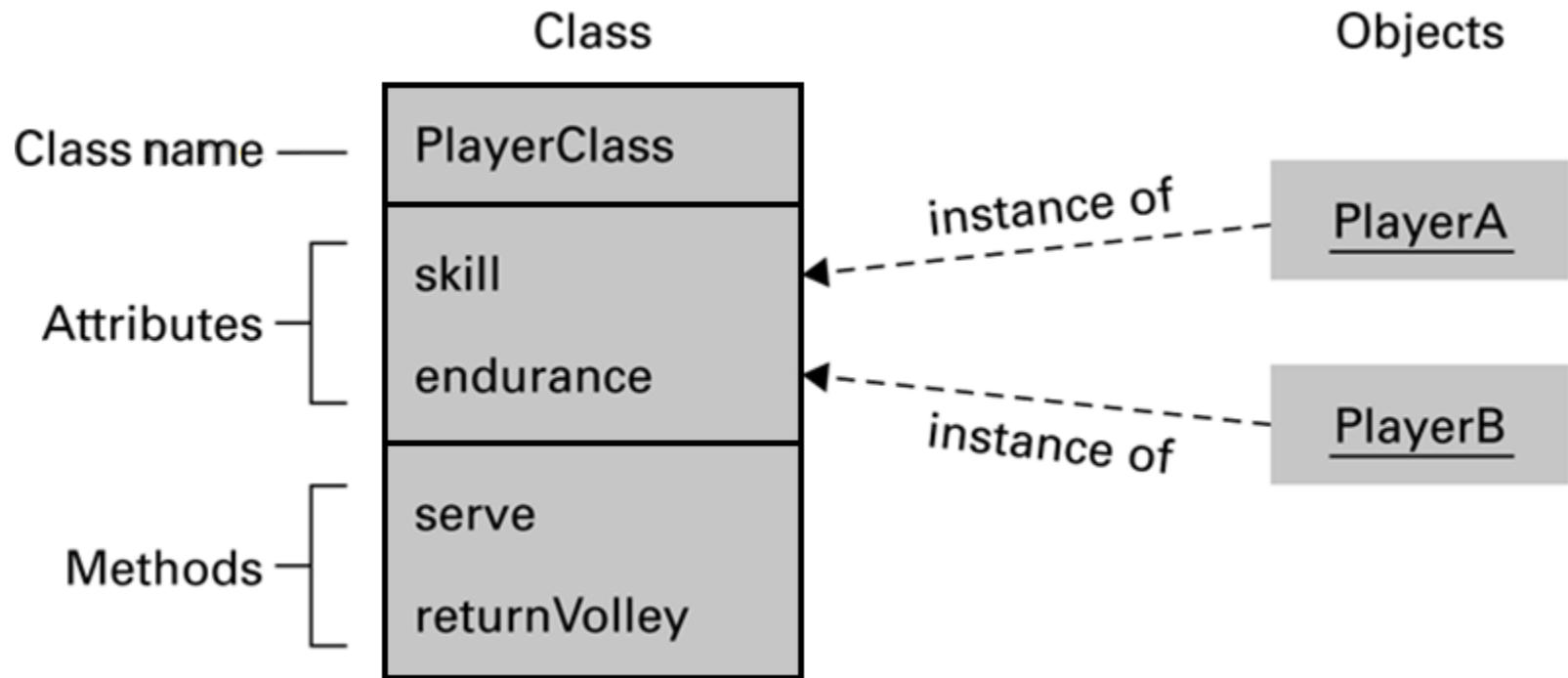
- **Modularity** is the division of software into manageable units, referred as **modules**, each of which deals with only a part of the software's overall responsibility.
 - Functions – Imperative paradigm
 - Structure charts
 - Objects – Object-oriented paradigm
 - Object collaboration diagrams
 - Components – reusable software units, often for object-oriented paradigm
 - Component interfaces

Tennis Game: Structure Chart for Imperative Paradigm

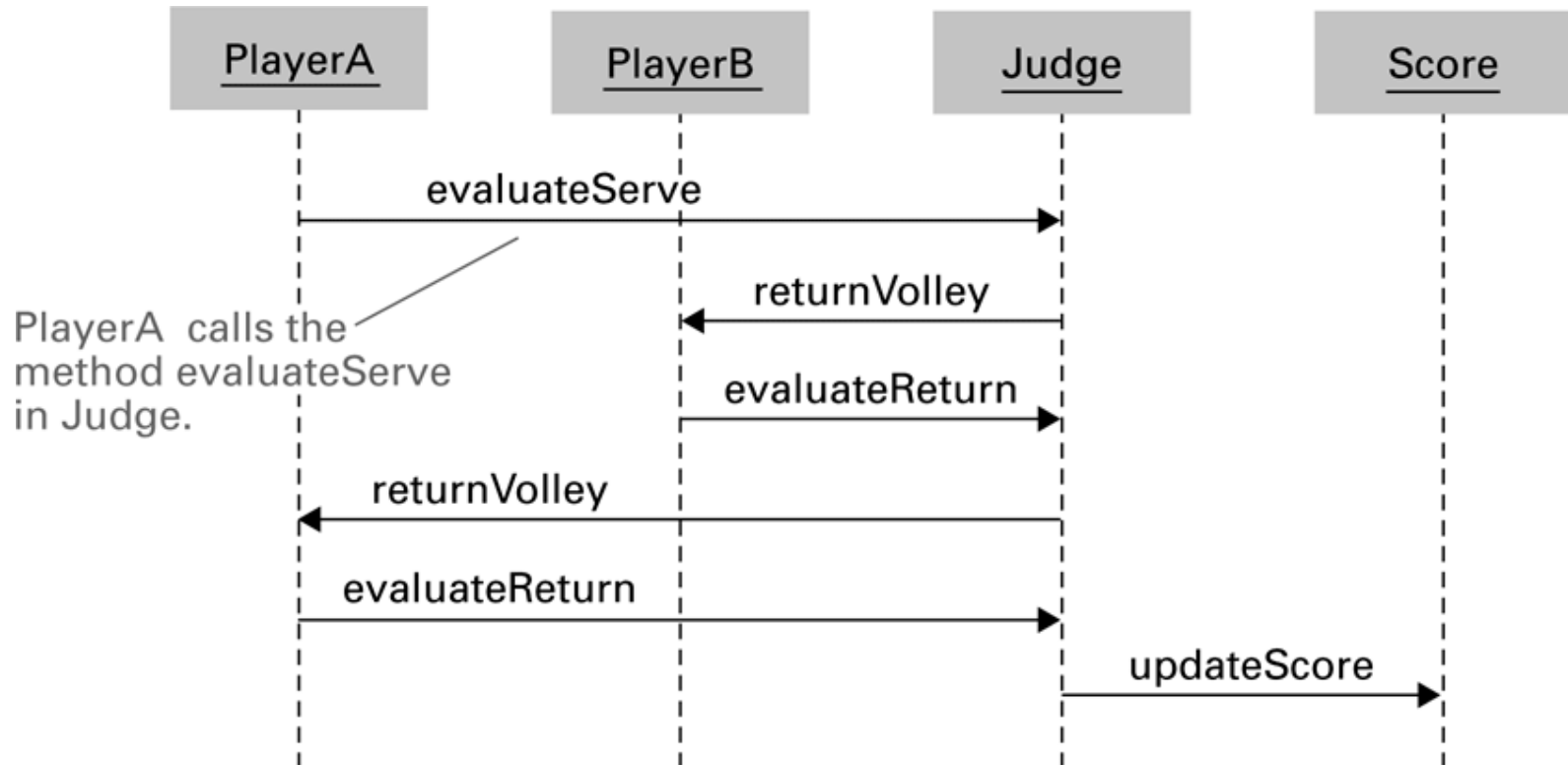
- ControlGame() procedure depends on several other procedures
 - Serve() – serves the ball to the game
 - Return() –returns the ball
 - ComputePath() – computes the trajectory of the ball
 - UpdateScore() – after miss is reported, updates the score



PlayerClass diagram and its instances for OO paradigm



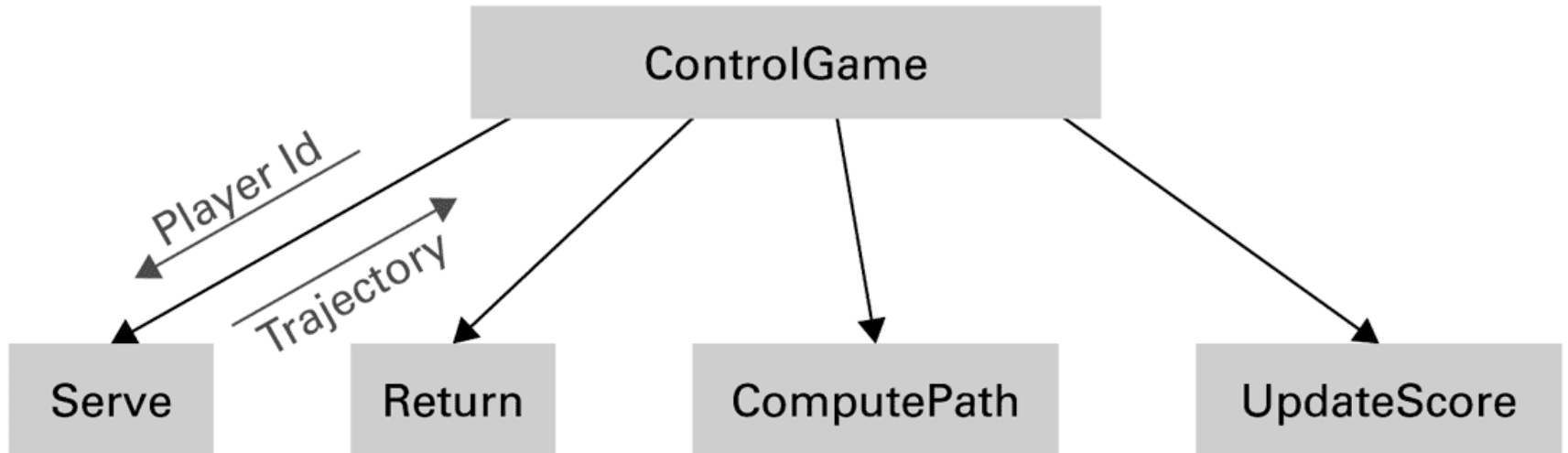
The interaction between objects resulting from PlayerA's serve for OO paradigm



Coupling

- Control coupling
 - One module passes control to another
- Data coupling
 - Sharing of data between modules
 - Global data is shared implicitly by all modules and may cause access problems.
- **Coupling increases between two classes A and B if:**
 - A has an attribute that refers to (is of type) B.
 - A calls on methods of an object B.
 - A has a method that references B (via return type or parameter).
 - A is a subclass of (or implements) class B.

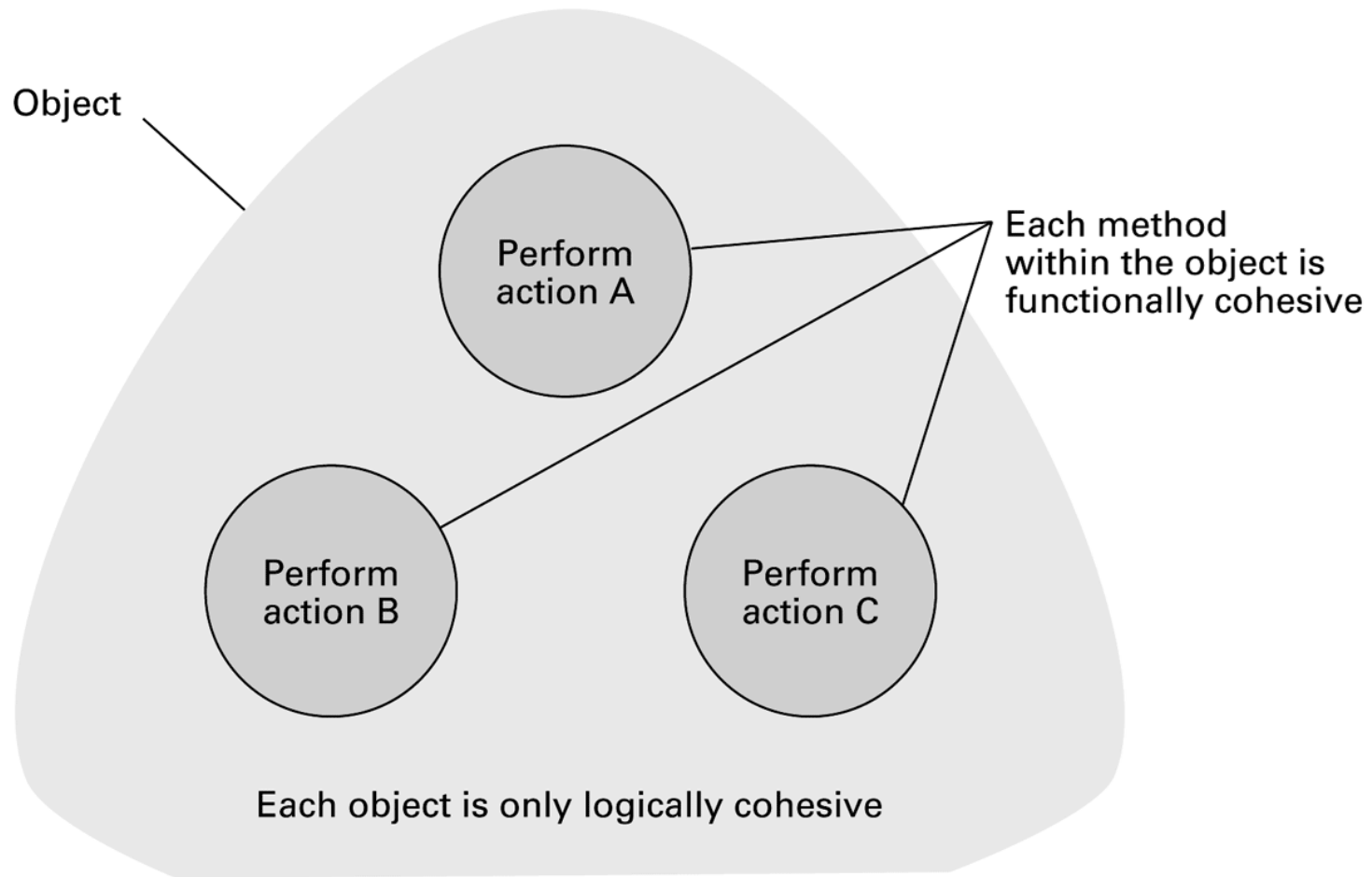
A structure chart including data coupling



Cohesion

- Logical cohesion
 - Logical similarity of actions and components in a module
- Functional cohesion
 - Each component focus on performing a single activity
 - Stronger than logical cohesion
- Cohesion in OO systems
 - Entire object (i.e. the collection of data fields and methods in an object) should be logically cohesive
 - Each method (i.e. the tasks you perform in a method should be functionally cohesive)
- Cohesion is increased if:
 - The functionalities embedded in a class, accessed through its methods, have much in common.
 - Methods carry out a small number of related activities, by avoiding coarsely grained or unrelated sets of data.
 - Related methods are in the same source file or otherwise grouped together; for example, in separate files but in the same sub-directory/folder.

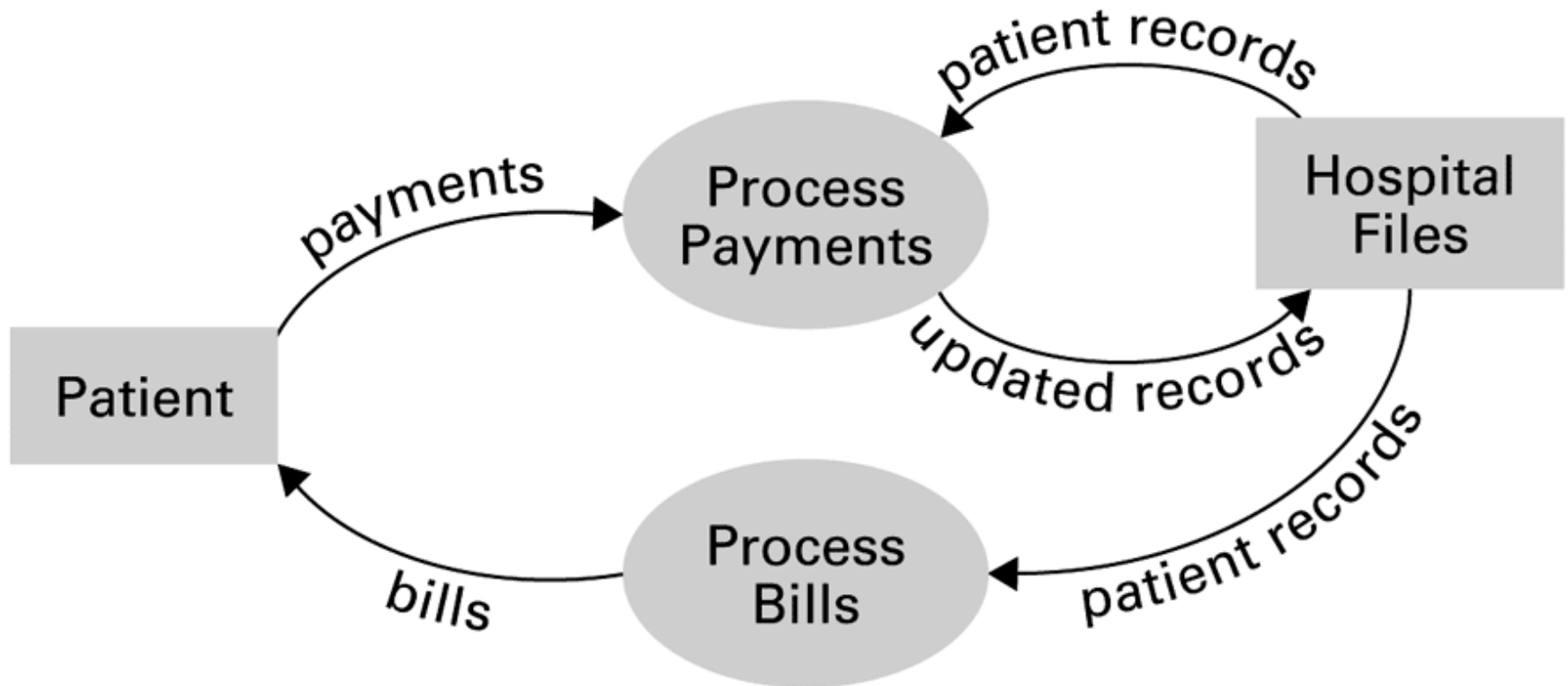
Logical and functional cohesion within an object



Tools of the Trade

- Data Flow Diagram
- Unified Modelling Language (UML)
- Entity-Relationship Diagram
 - One-to-one relation
 - One-to-many relation
 - Many-to-many relation
- Data Dictionary

A simple dataflow diagram

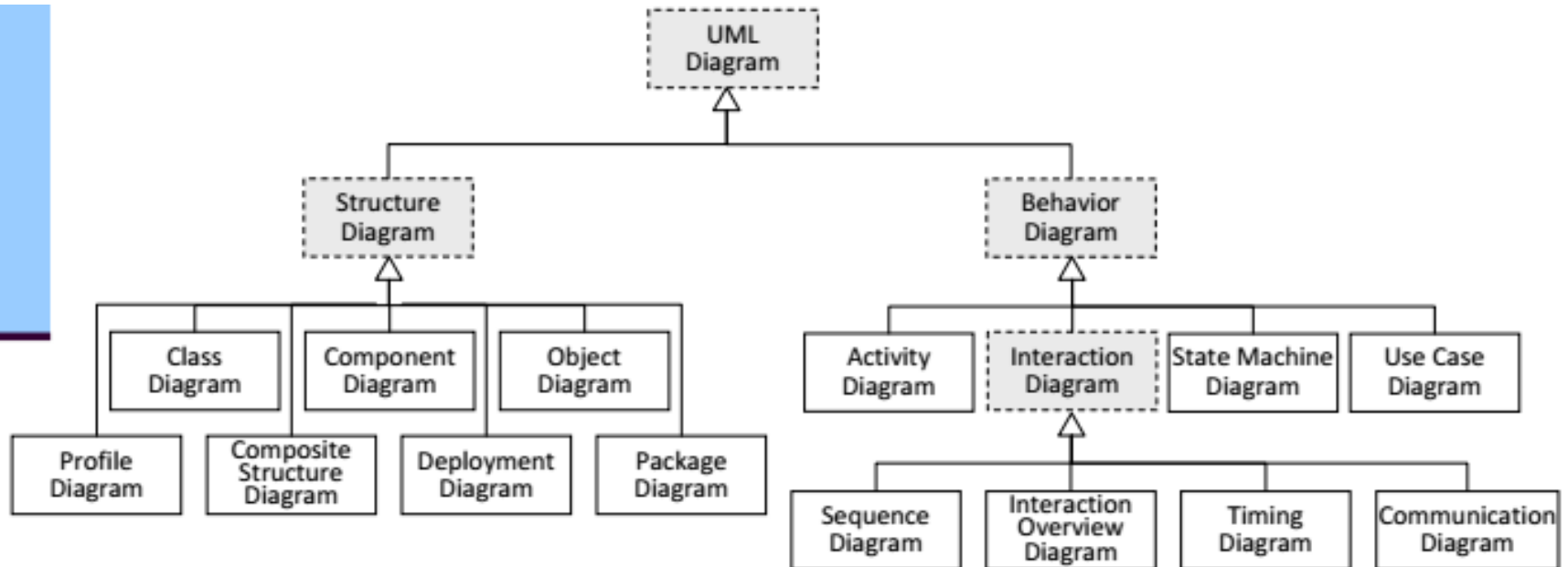


UML: Unified Modelling Language

- UML is a language for specifying, visualizing, constructing and documenting large complex systems
- The development of UML began in late 1994 by Grady Booch and Jim Rumbaugh of Rational Software Corporation
- UML 1.0, and later 1.1 was officially released in 1997.
- UML 2.0 is adopted in 2005

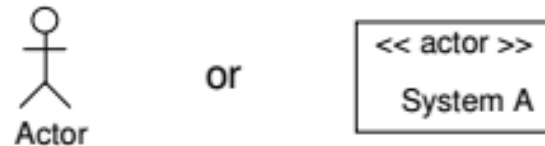
UML Diagrams

- There are 14 types of UML 2.0 diagrams:

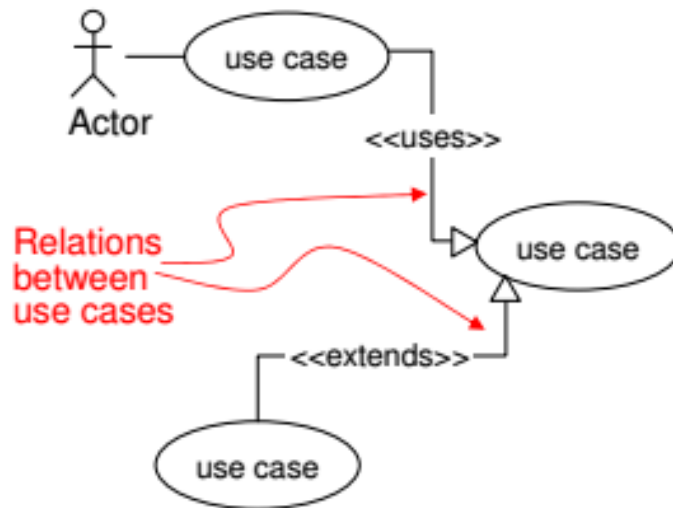


Use Case Diagram

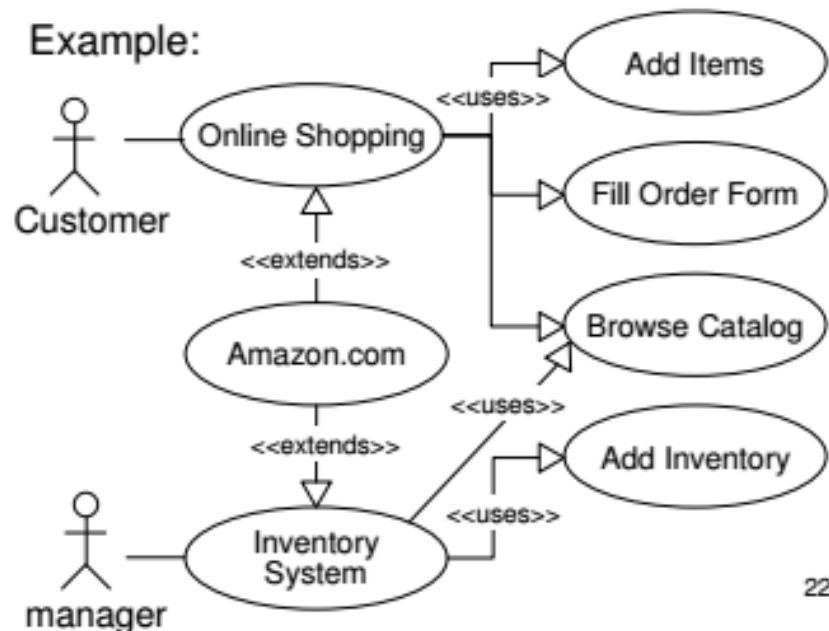
- ❑ Actors in UML are the users of a system:



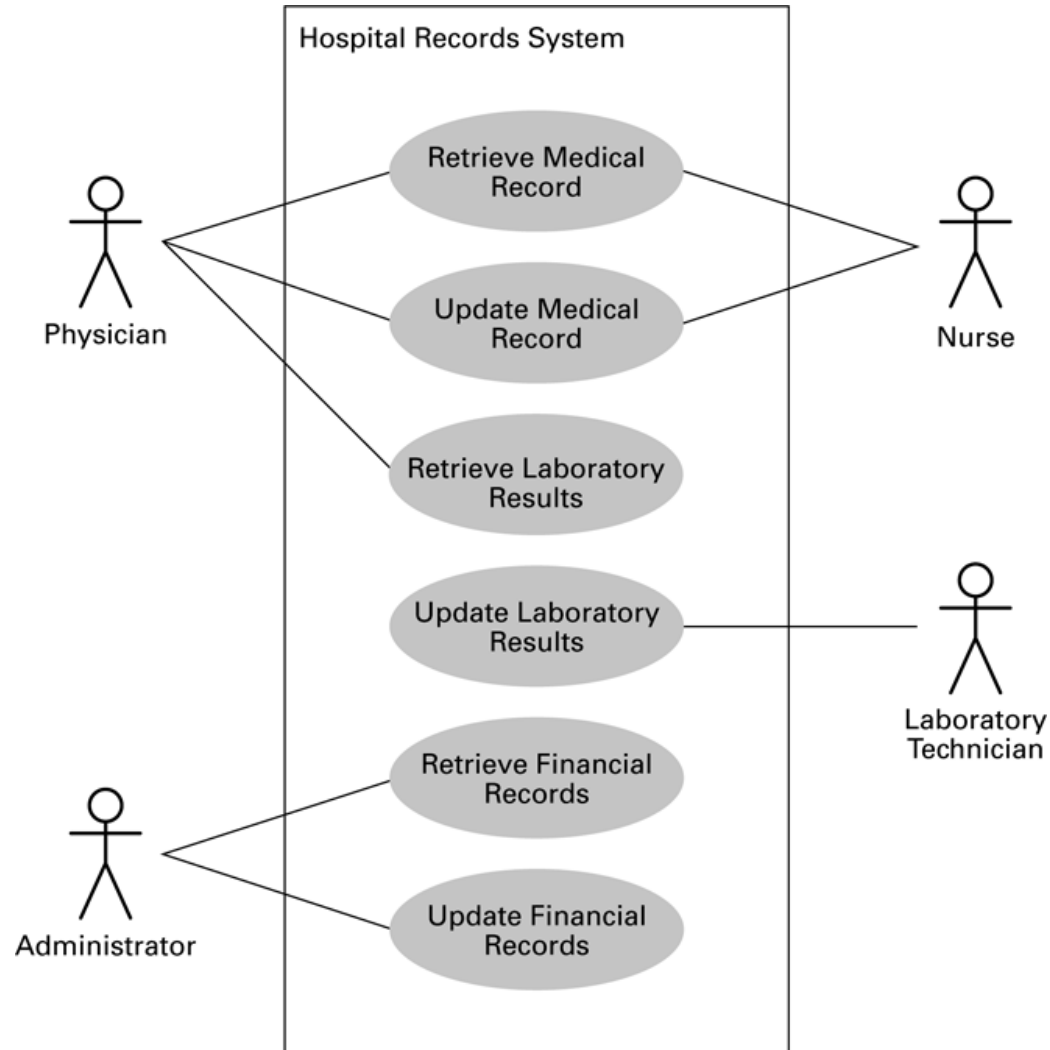
- ❑ Use case diagrams model the functions of systems



Example:

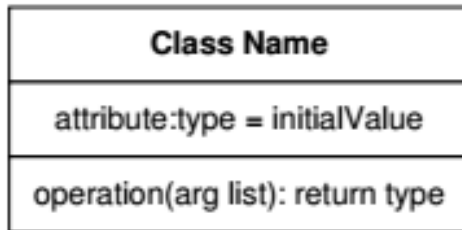


A simple use case diagram



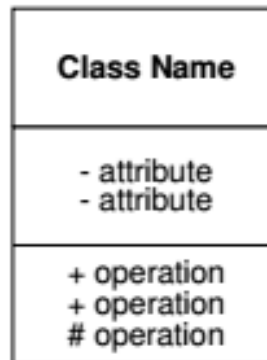
Class Diagram

- ❑ Class diagrams describe the relations (dependencies or collaborations) among different classes
- ❑ Class definition:



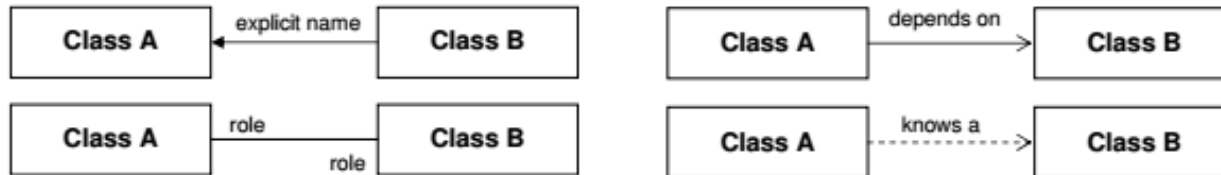
- ❑ Visibility

- + means public
- - means private
- # means protected



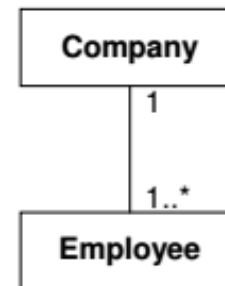
Class Diagram 2

□ Association



□ Multiplicity

- 1 means no more than one
- 0..1 means zero or one
- * means many
- 0..* means zero or many
- 1..* means one or many

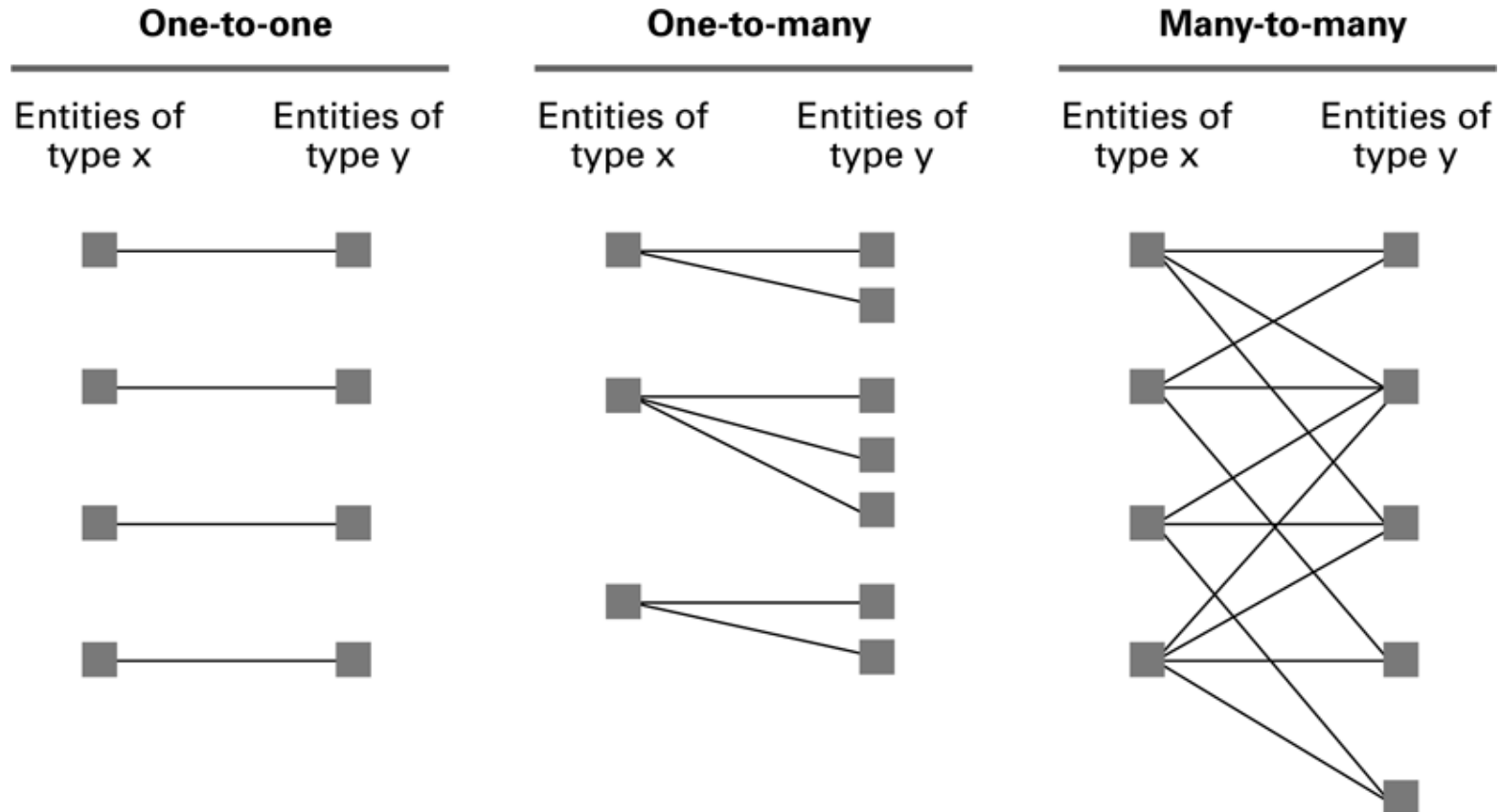


25/38



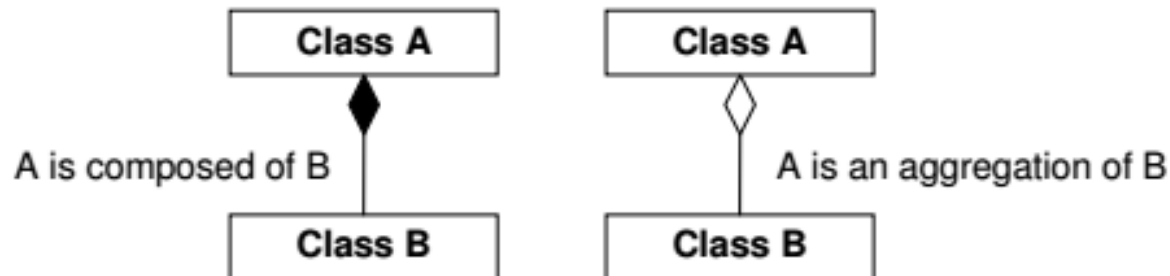
Relationships between Entities

- It is important to show the cardinality of the relationships between classes (entities in databases context).
- Entity-Relationship Diagram commonly used by Database Designers shows that.

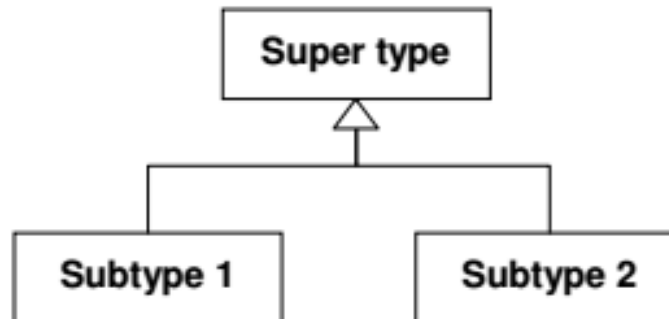


Class Diagram 3

❑ Composition and aggregation



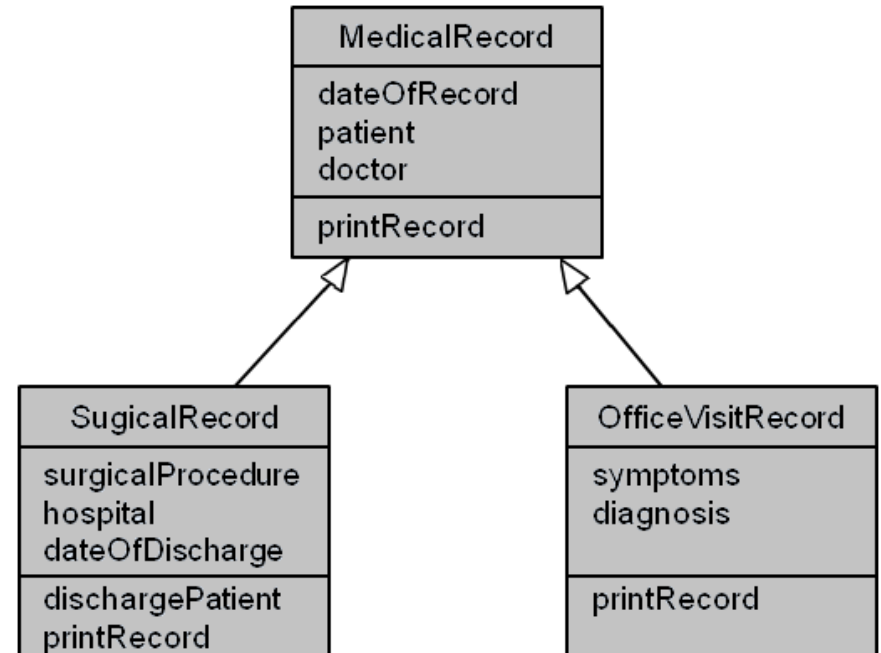
❑ Generalization (inheritance)



A class diagram depicting generalization

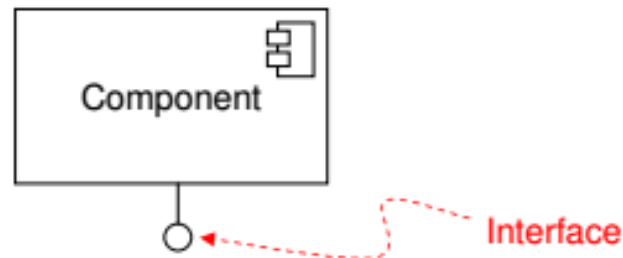
The concept of “inheritance” in OO is called “generalization” in UML.

- In inheritance, child classes contain additional attributes and methods as well as the public/protected attributes and methods of their parent.



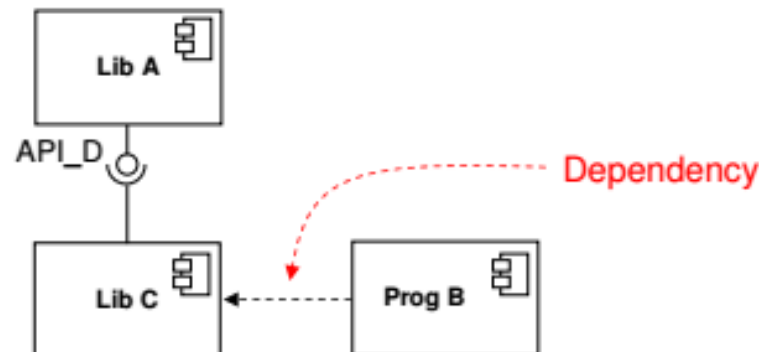
Component Diagram

- ❑ A component is a building block of the system



- ❑ Dependency of components

- Example: Program B depends on library C which uses library A through interface API_D can be expressed as follows,

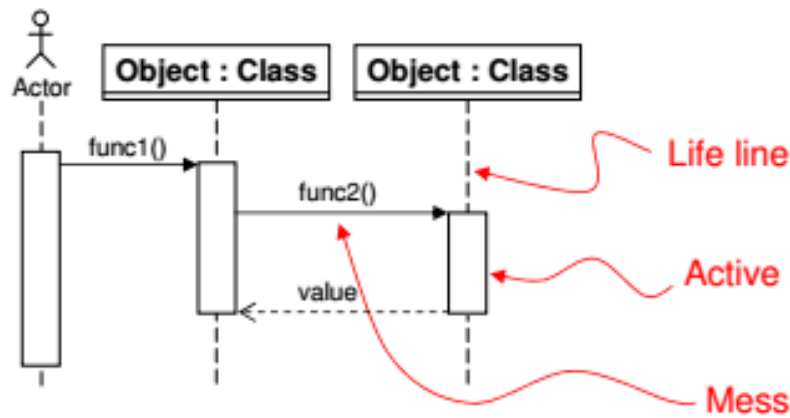


Sequence Diagram

- Sequence diagrams present the flow of messages between instances, objects, or processes along time
- The time-axis of a sequence diagram progresses in the vertical direction
- Messages flow can be synchronous (blocking) or asynchronous (non-blocking)

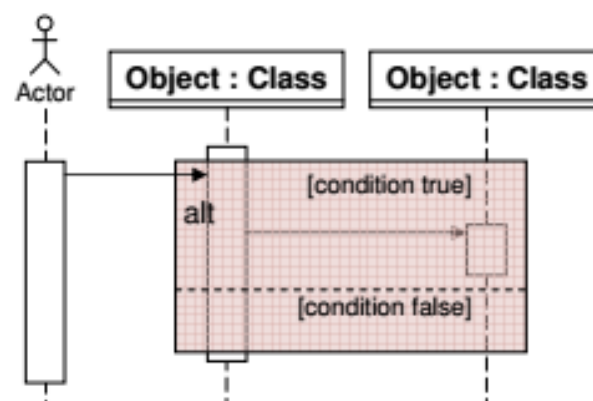
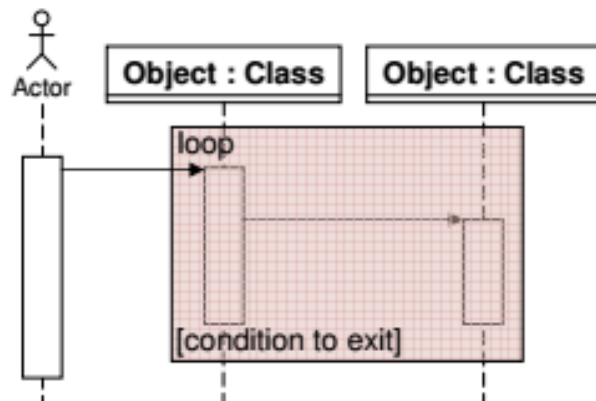
Sequence Diagram 2

□ Call sequence



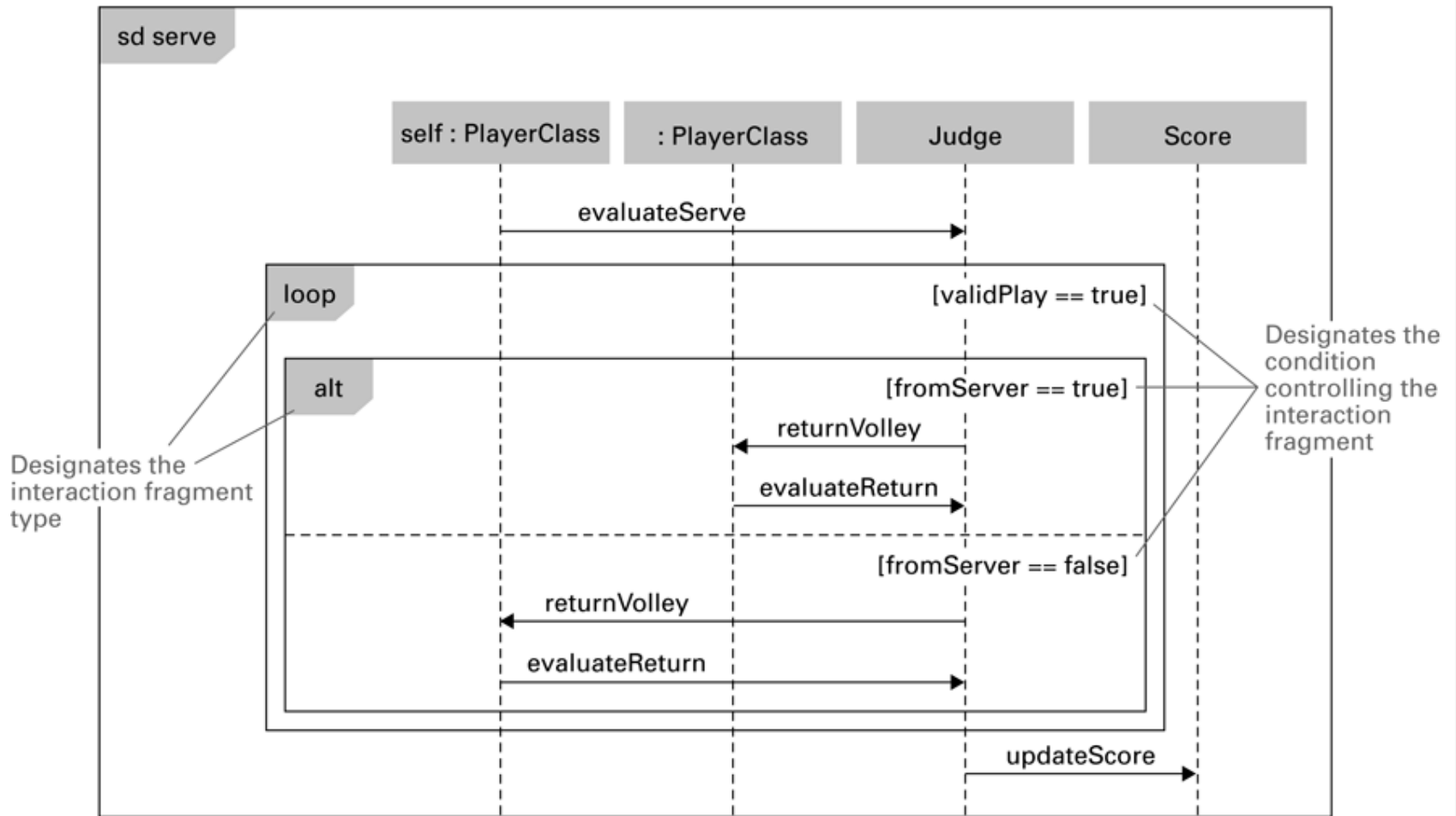
Arrow	Message type
	Synchronous call
	Generic flow of control
	Asynchronous flow of control
	Reply
	Time out call (non-standard)

□ Loop and branches



30/38

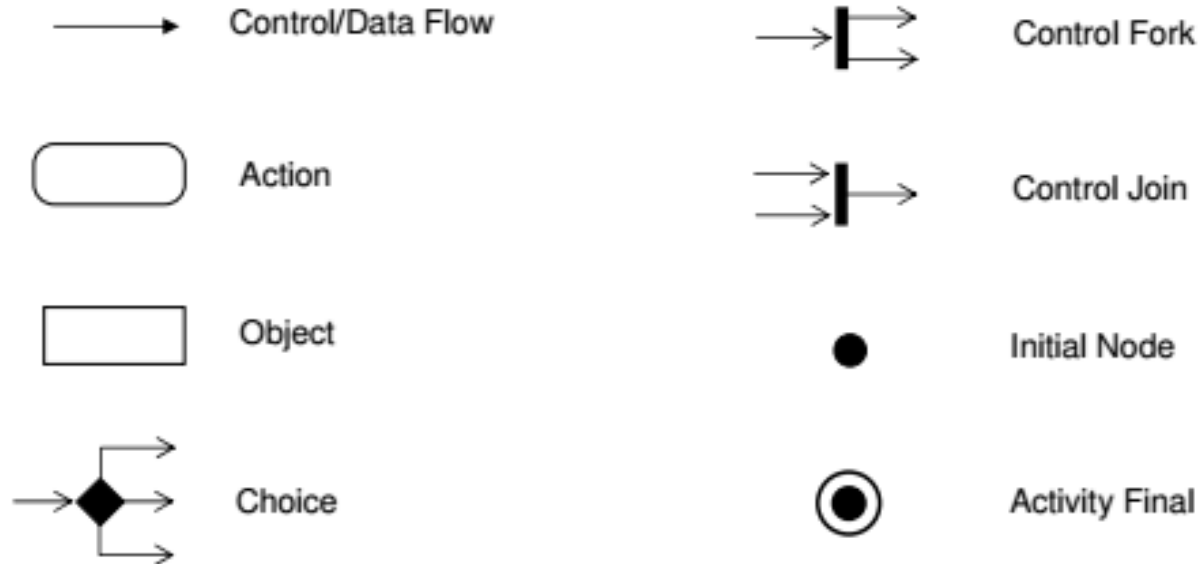
A sequence diagram depicting a generic volley



Activity Diagram (Flow chart)

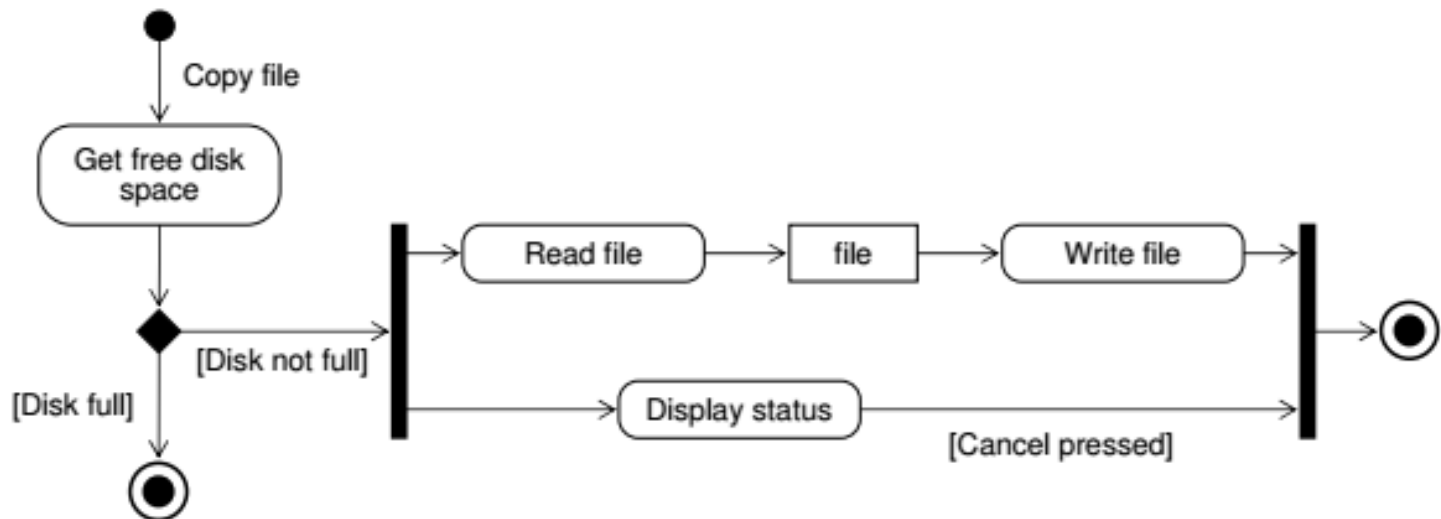
□ Activity diagrams show flow of control and data flow

■ Notations:



Example of Activity Diagram

- ❑ File copy activity can be illustrated as follows:

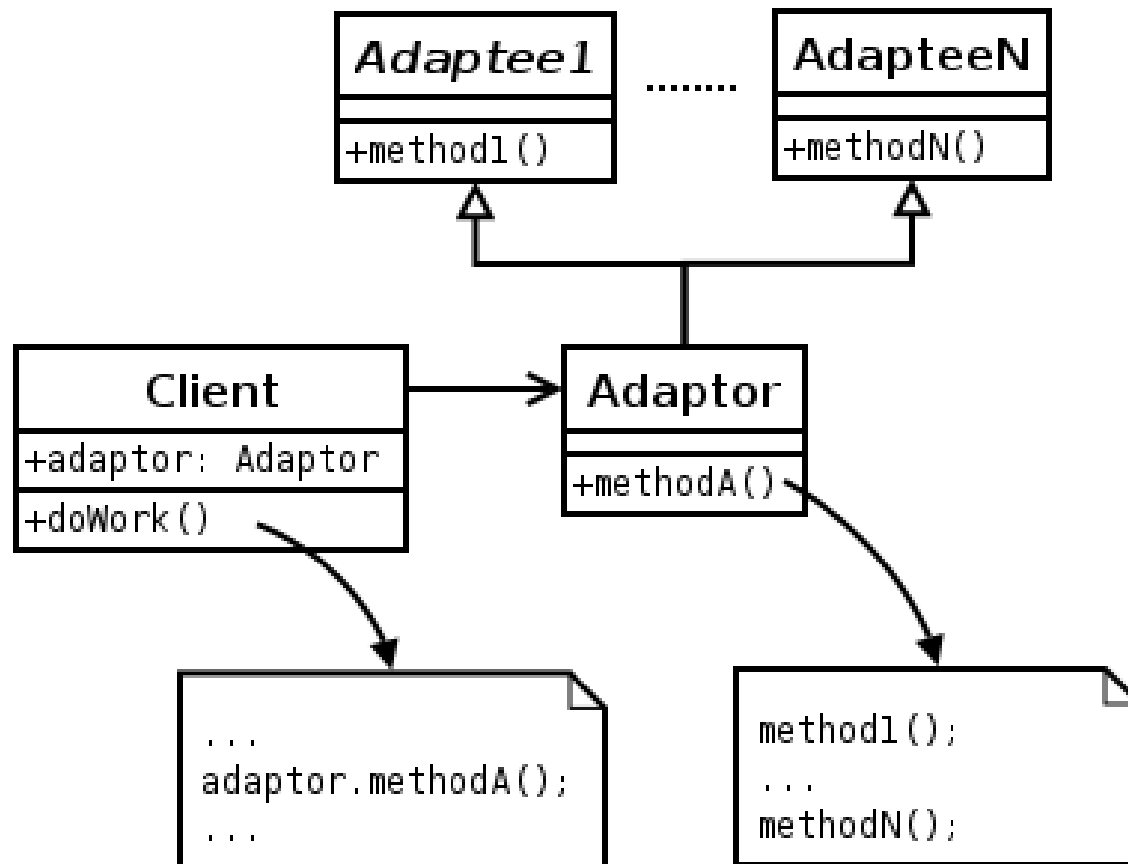


Design Patterns

- Well designed “templates” for solving recurring problems
- Examples:
 - Adapter pattern: Used to adapter a module’s interface to current needs
 - Decorator pattern: Used to control the complexity involved when many different combinations of the same activities are required
- Inspired by the work of Christopher Alexander in architecture

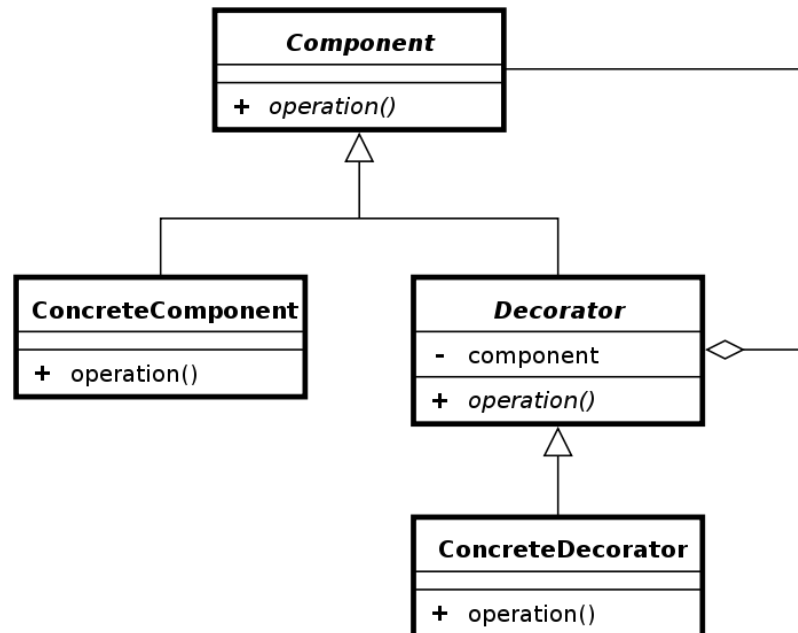
Adapter Pattern

- The Adapter design pattern allows otherwise incompatible classes to work together by converting the interface of one class into an interface expected by the clients.



Decorator Pattern

- The decorator pattern can be used to extend (decorate) the functionality of a certain object statically, or in some cases at run-time, independently of other instances of the same class, provided some groundwork is done at design time.



Software Testing Strategies

- Glass-box testing
 - Pareto principle – typically, only a small amount of software modules in a large system are problematic
 - Basis path testing – test data should enable testing of all possible execution branches in a software system
- Black-box testing
 - Boundary value analysis (stress testing) – trying out the minimum and maximum values in permissible range of values.
 - Beta testing – giving the software to small portion of end-users to test and receive their feedback. (Alpha testing is done by the developers themselves).

Documentation

- User Documentation
 - Printed book for all customers
 - On-line help modules
- System Documentation
 - Source code is part of the system documentation
 - Consistent coding style and naming conventions
 - Comments
 - Design documents – requirement specifications, program descriptions (in UML)
- Technical Documentation
 - For installing, customizing, updating, etc.

Computer Aided Software Engineering (CASE) tools

- Project planning – for resource (time, personnel) allocation
- Project management – for status tracking
- Documentation – for semi-automatic document generation (see Doxygen)
- Prototyping and simulation – for fast proof-of-concept
- Interface design – for GUI development
- Programming – for program coding, version control and debugging

Software Ownership

- Copyright of software
 - Allow a product to be released while retaining ownership of intellectual property
 - Asserted in all works:
 - Specifications
 - Source code
 - Final product
- Software License
 - A legal agreement that grants the user certain permissions without transferring ownership
- Patents
 - Must demonstrate that it is new, usable, and not obvious to others with similar backgrounds
 - Process is expensive and time-consuming

Suggested Activity/Reading

- Read Book about Design Patterns:
 - E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, ISBN 0-201-63361-2, 1995.
- Check more examples in UML:
 - <http://creately.com/blog/diagrams/uml-diagram-types-examples>
 - <http://www.tutorialspoint.com/uml/>
- Learn more about SCRUM:
 - <http://www.scrumguides.org/scrum-guide.html>